

Model Context Protocol (MCP) — Complete Notes

AI Agents ke liye USB-C Port — Roman Urdu mein Asaan Notes

Prepared for:

AI Engineer | Machine Learning Engineer | Generative AI / Agentic AI Engineer

Is document mein kya hai:

- MCP kya hai aur kyun banaya gaya — asaan Roman Urdu explanation
- MCP ke bina agentic apps mein kya masla hota hai (real problem)
- MCP Client, MCP Server, Tools aur LLM ka reasoning flow
- JSON-RPC 2.0 message format aur Transport types (STDIO / Streamable HTTP)
- Local vs Remote vs Custom MCP Server ka farq
- Practical implementation walkthrough — TripMate AI project ke sath
- Har diagram original slides se extract kar ke laga diya gaya hai

Source: MCP tutorial video transcript + Excalidraw diagram file (as uploaded)

Table of Contents

1. MCP Kya Hai? (Introduction)
2. MCP Ke Bina Masla — Problem Without MCP
3. MCP Ka Solution — Using MCP
4. MCP Ke Core Components (Client + Server + Tools)
5. Ek Client, Multiple Servers (Real Example)
6. Communication Protocol — JSON-RPC 2.0
7. Types of MCP Servers — Local, Remote, Custom
8. Transport Types — STDIO vs Streamable HTTP
9. Practical Implementation — TripMate AI Project
10. Benefits Summary (Without MCP vs With MCP)
11. Extra Additions — Agentic AI Engineer ke liye Best Practices

SECTION 1

MCP Kya Hai? (What is MCP)

MCP ka matlab hai **Model Context Protocol**. Ye ek **universal USB-C port** ki tarah kaam karta hai, lekin AI applications ke liye. Jaise USB-C port ek hi connector se har device (laptop, phone, camera) ko connect kar deta hai, waise hi MCP ek hi standardized connection se AI model ko kisi bhi tool, API, ya database ke sath jorr deta hai.

Simple lafzon mein: **Custom API integrations banane ki bajaye**, jaise GitHub, Google Drive, weather API, flight API, ya local database ke sath — developers ab MCP use karte hain taake ek **standardized aur reusable connection** ban jaye.

Yaad rakhne wali baat (Key Analogy)

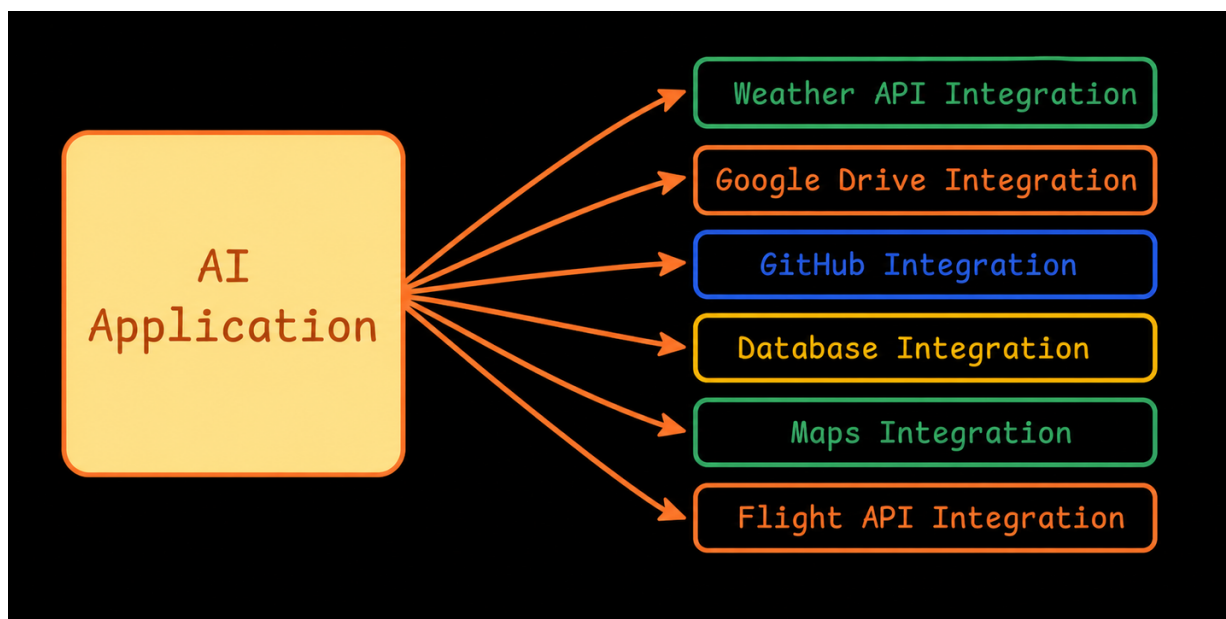
MCP = "Universal USB-C Port for AI Applications"

Jis tarah USB-C se aap kisi bhi device ko connect kar sakte hain bina alag alag cable ke, usi tarah MCP se AI agent kisi bhi tool/API/database ko bina alag custom code likhe access kar sakta hai.

SECTION 2

MCP Ke Bina Masla — Problem Without MCP

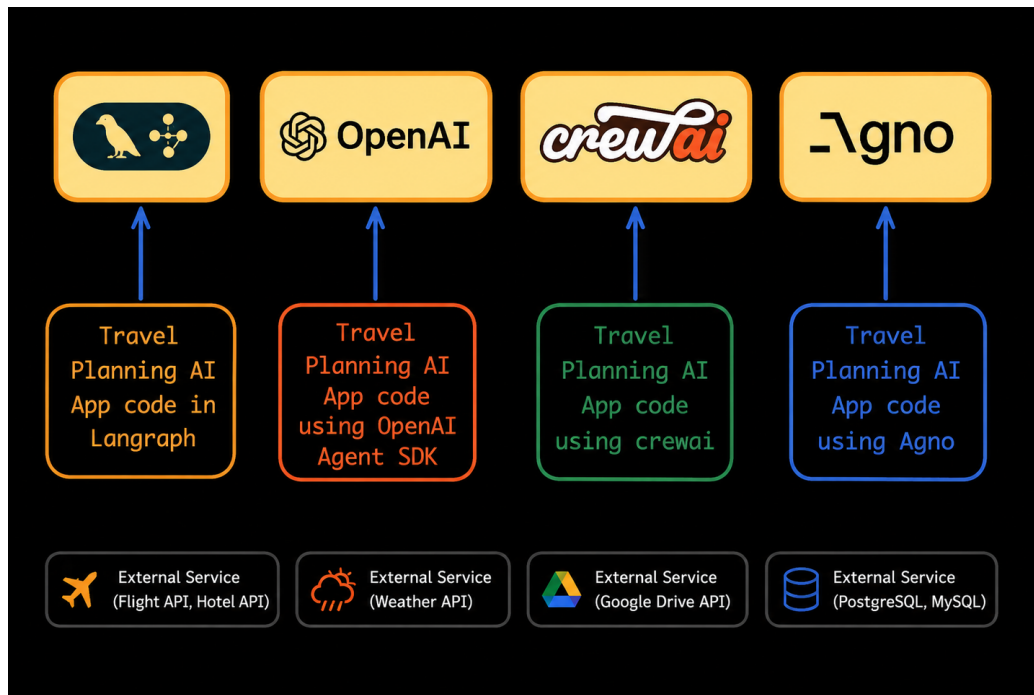
Video mein ek real project ka example diya gaya hai: **TripMate AI** — ek multi-agent travel planner jo LangGraph se banaya gaya. Is app mein 4 agents thay: Flight Agent, Hotel Agent, Itinerary Agent, aur Final Response Agent. Har agent kisi na kisi **custom function (tool)** se kaam karta tha — jaise Aviation Stack API (flights ke liye) aur Tavly API (real-time search ke liye).



AI Application ko har external service (Weather, Google Drive, GitHub, Database, Maps, Flight API) ke sath alag-alag custom API integration likhni parti hai.

Is approach ke 2 bade masle (problems):

- **Masla 1 — API Version Change:** Agar aap ne apni custom function mein API ka version 1 (v1) use kiya hai aur company kal ko us API ko version 2 (v2) mein upgrade kar de, to aapki application break ho jayegi. Aapko manually code khol kar URL/logic update karna paray ga. Agar aap ke paas hazaron tools hon to har API ki documentation roz check karna namumkin hai.
- **Masla 2 — Framework Lock-in:** Agar aap ne LangGraph mein ek custom tool likha hai aur kal ko CrewAI ya OpenAI Agent SDK par switch karna paray, to har framework ke liye alag se wahi tool dobara likhna parta hai.



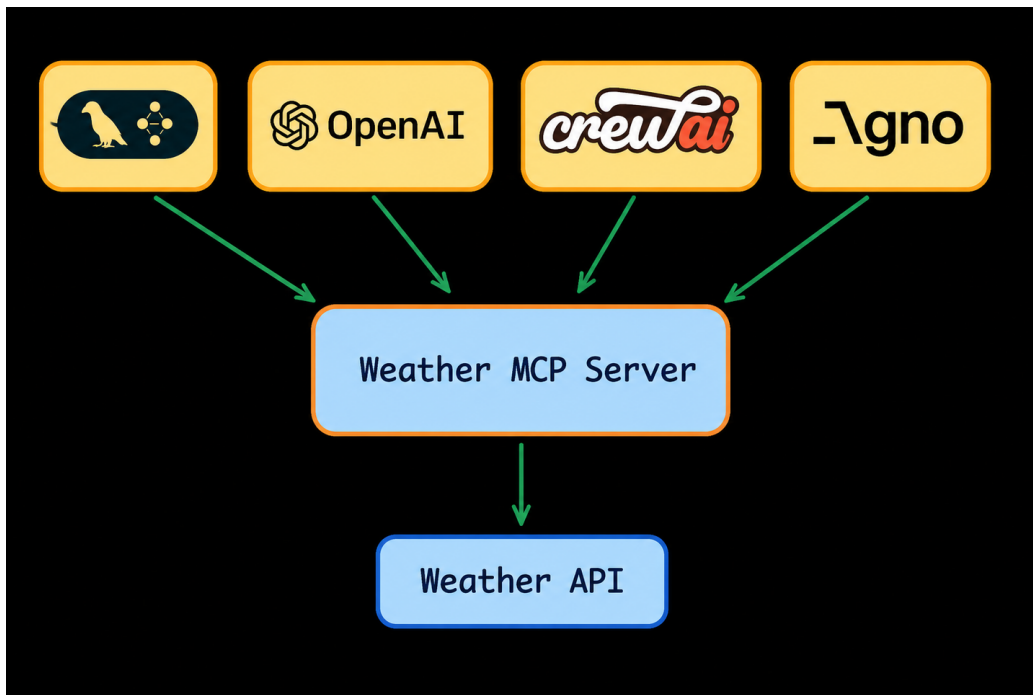
Har AI framework (LangGraph, OpenAI, CrewAI, Agno) ke liye alag se Travel Planning App code likhna parta hai — bina MCP ke ye repetition (duplication) hoti hai.

Yahi wo jaga hai jaha application ko **"self-healing"** hona chahiye — matlab agar underlying API change ho, to app ko khud automatically adjust ho jana chahiye, developer ko har baar manually fix karne ki zaroorat na pary. Isi zaroorat ki wajah se researchers ne MCP introduce kiya.

SECTION 3

MCP Ka Solution — Using MCP

Ab custom function ki jaga, hum ek **MCP Server** use karte hain jo already kisi API/tool ke sath connected hota hai. Ye MCP Server kisi bhi framework (LangGraph, OpenAI, CrewAI, Agno) ke sath connect ho sakta hai — **ek hi standardized connection** ke zariye.



Har framework (LangGraph, OpenAI, CrewAI, Agno) ek hi Weather MCP Server se connect ho sakta hai, aur ye server aage Weather API se baat karta hai.

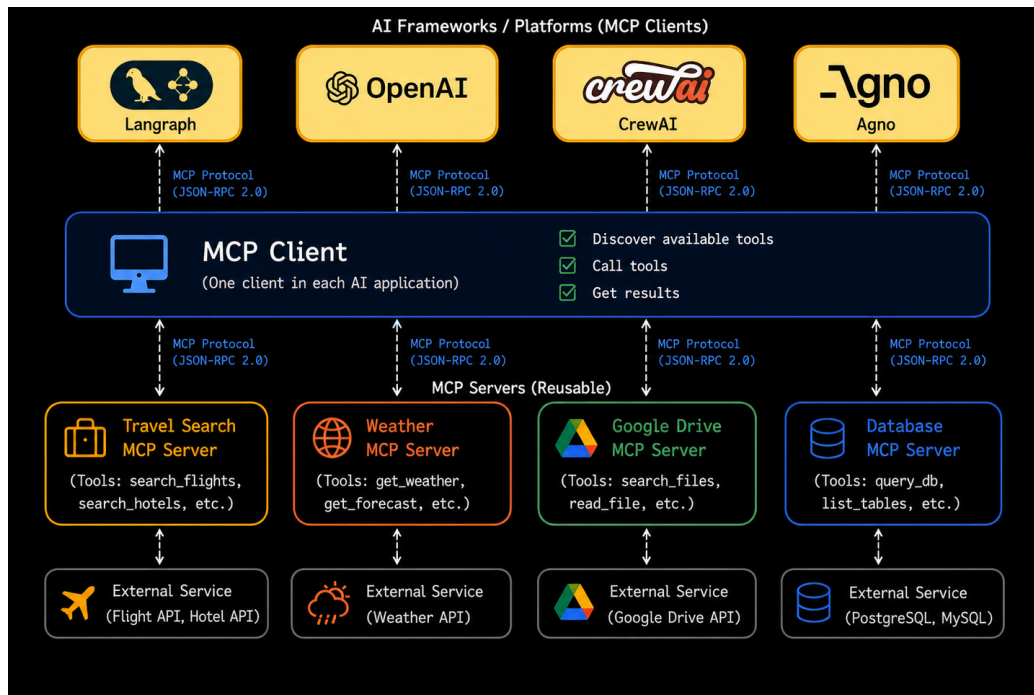
Is se kya faida hota hai?

- Agar Weather API v1 se v2 mein change ho jaye, to sirf **MCP Server ke andar** update hota hai — aapki app ka code bilkul nahi badalta.
- Ye connection **framework-independent** hai — LangGraph se CrewAI switch karo, MCP connection wahi rehta hai.
- Third-party MCP servers khud apni API updates handle karte hain — **koi downtime nahi hota**, user ko kabhi bhi "data nahi mil raha" wala bura experience nahi milta.

SECTION 4

MCP Ke Core Components

MCP ke do major components hote hain: **MCP Client** aur **MCP Server**. Ye dono mil kar poore MCP ecosystem ko chalte hain.



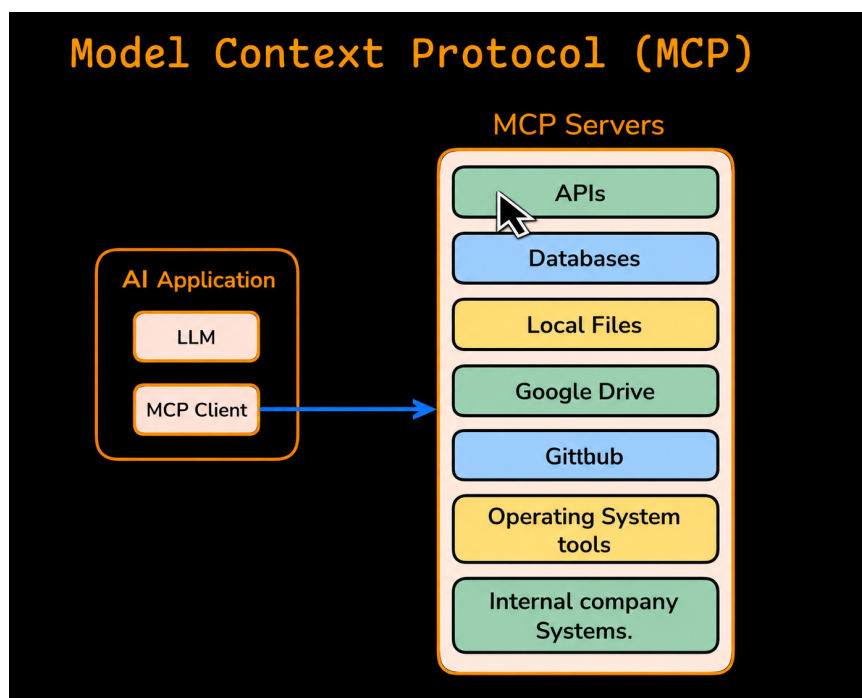
Poori MCP Architecture: AI Frameworks (Clients ki taraf) → MCP Client → MCP Protocol (JSON-RPC 2.0) → Reusable MCP Servers (Travel, Weather, Google Drive, Database) → External Services / APIs.

MCP Client

- Har AI application (jo LangGraph, CrewAI, OpenAI, ya Agno se bani ho) ke andar ek **MCP Client** hona zaroori hai agar wo MCP Server use karna chahti hai.
- MCP Client ka kaam hai: **available tools discover karna, tools ko call karna, aur results wapas lana.**
- MCP Client, MCP Server aur AI Agent ke darmiyan ek **bridge** ki tarah kaam karta hai.

MCP Server

- Ek MCP Server kisi specific domain ke **tools ka group** hota hai — jaise Weather MCP Server (get_weather, get_forecast), Google Drive MCP Server (list_files, read_file, create_file), Database MCP Server (query_db, list_tables).
- Ye server internally kisi **external API/service** se connected hota hai.
- MCP Server **reusable** hota hai — ek dafa ban gaya to kisi bhi framework ke MCP Client se use ho sakta hai.



AI Application ke andar LLM + MCP Client hota hai, jo MCP Servers ke through APIs, Databases, Local Files, Google Drive, GitHub, OS tools, aur Internal Company Systems tak access kar sakta hai.

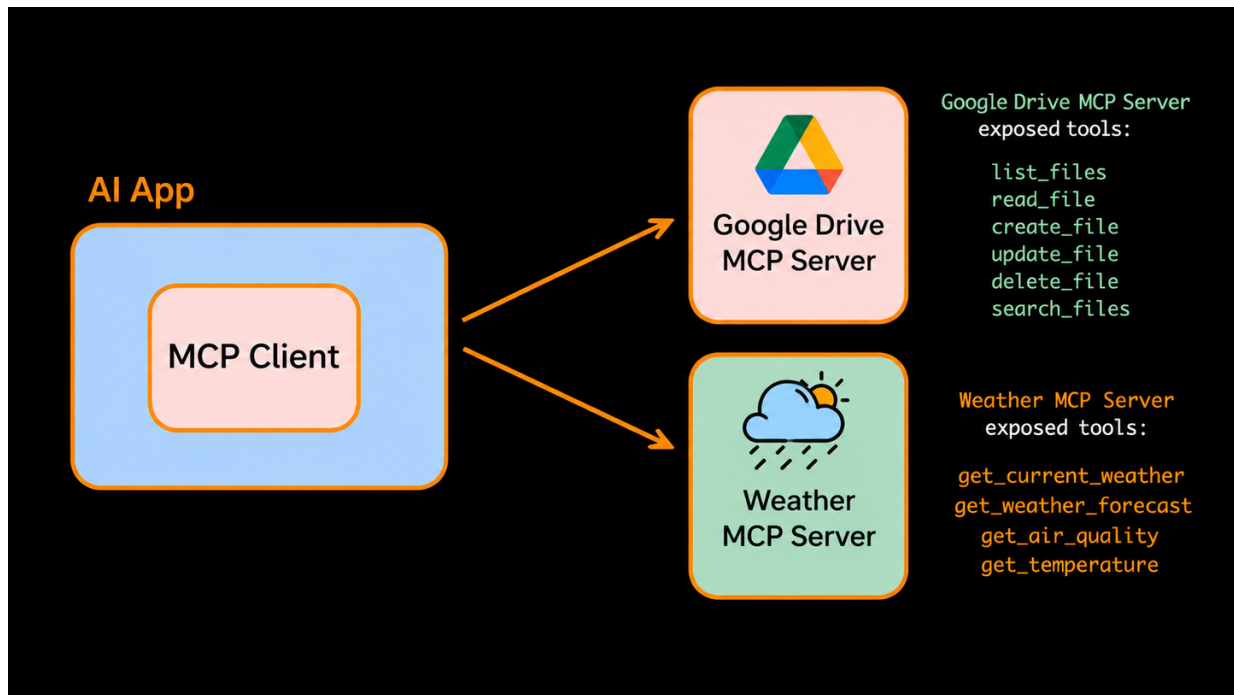
LLM Ka Reasoning Role

Har agent ke andar ek **LLM (Large Language Model)** hota hai jis mein reasoning power hoti hai. Jab user koi query deta hai (jaise "Delhi ka weather kya hai?"), to LLM khud decide karta hai ke kaun sa MCP Server aur kaun sa tool use karna hai. Ye poora decision-making automatically hota hai — developer ko manually if-else likhne ki zaroorat nahi.

SECTION 5

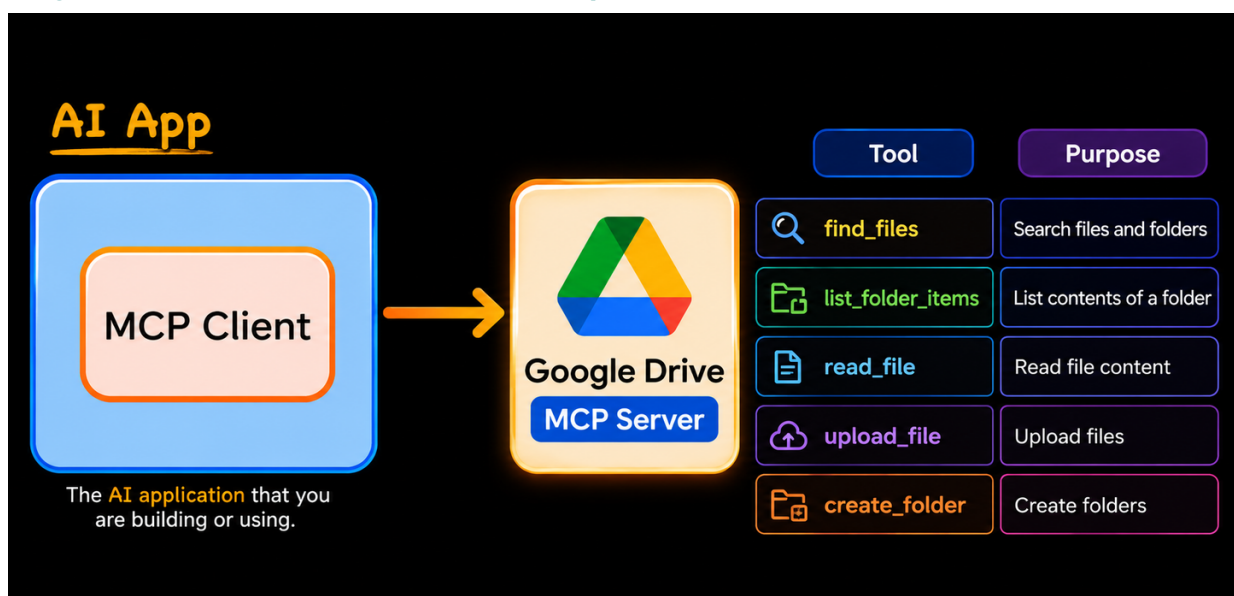
Ek Client, Multiple Servers (Real Example)

Ek MCP Client ek se zyada MCP Servers ke sath connect ho sakta hai. Neeche diya gaya example dikhata hai ke ek hi MCP Client, Google Drive MCP Server aur Weather MCP Server dono ko access kar sakta hai — har server ke apne separate exposed tools hote hain.



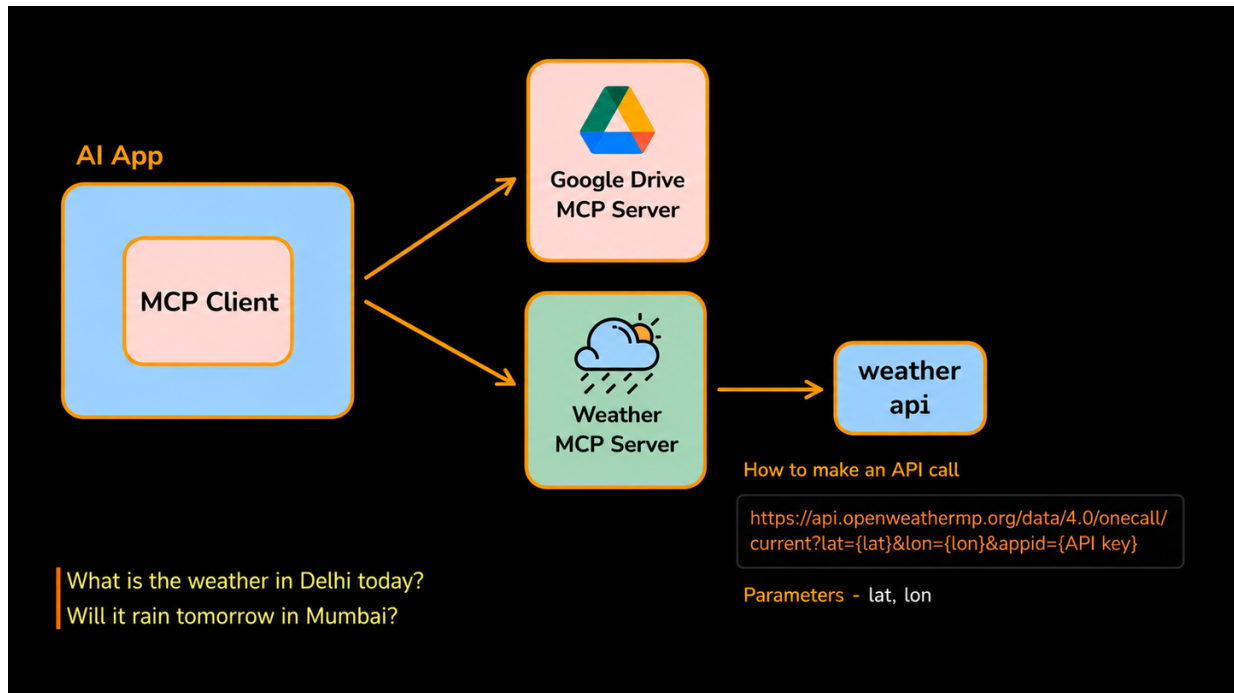
Ek MCP Client, Google Drive MCP Server (`list_files`, `read_file`, `create_file`, `update_file`, `delete_file`, `search_files`) aur Weather MCP Server (`get_current_weather`, `get_weather_forecast`, `get_air_quality`, `get_temperature`) dono ke sath connect.

Google Drive MCP Server — Tool aur Purpose Table



Har tool ka apna specific purpose hota hai: `find_files` (search), `list_folder_items` (list contents), `read_file` (read content), `upload_file` (upload), `create_folder` (create folder).

Jab user query karta hai "Delhi ka weather kya hai? Kal Mumbai mein baarish hogi?" — is waqt LLM samajhta hai ke **Weather MCP Server** use karna hai (na ke Google Drive), aur phir specific tool (get_current_weather) select karta hai. Weather MCP Server aage OpenWeather API ko call karta hai.

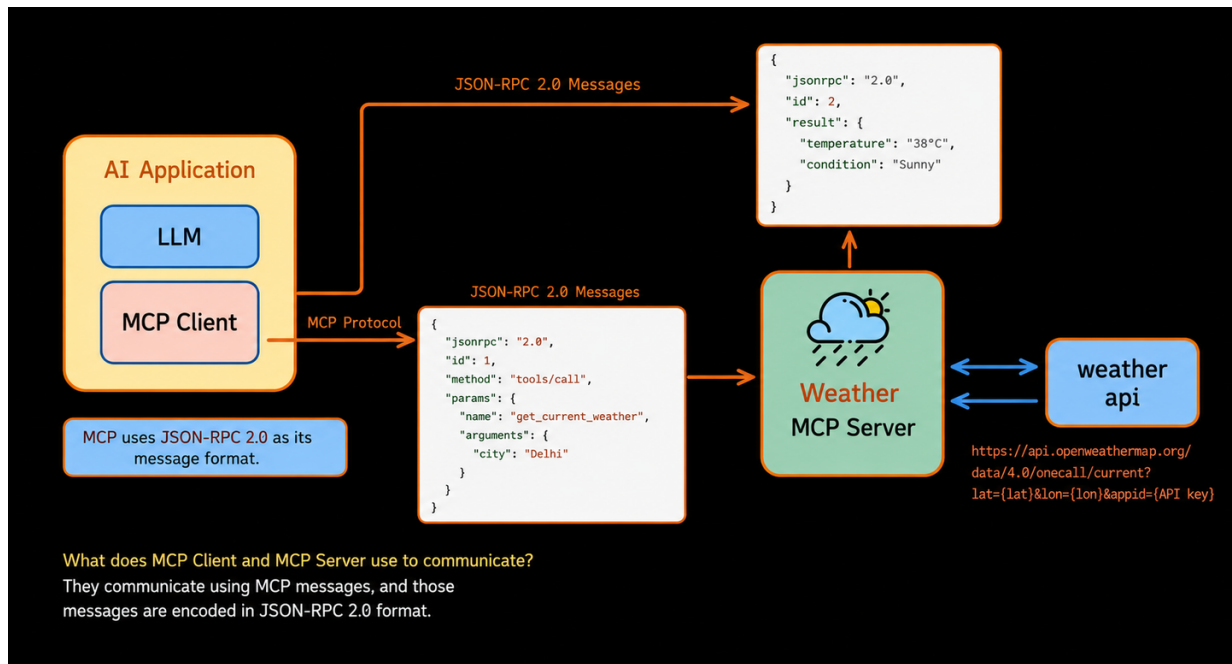


Weather MCP Server, Weather API ko call karta hai. Example URL:
`api.openweathermap.org/data/4.0/onecall/current?lat={lat}&lon={lon}&appid={API key}`

SECTION 6

Communication Protocol — JSON-RPC 2.0

MCP Client aur MCP Server aapas mein **MCP Protocol** ke zariye baat karte hain, jo **JSON-RPC 2.0** format mein messages exchange karta hai. Ye ek standardized message format hai jisme request/response dono JSON structure mein hote hain.



MCP Client se request JSON-RPC format mein jati hai (method: tools/call, params: city=Delhi) aur Weather MCP Server response bhi JSON-RPC format mein wapas bhejta hai (temperature, condition).

Request Example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_current_weather",
    "arguments": { "city": "Delhi" }
  }
}
```

Response Example:

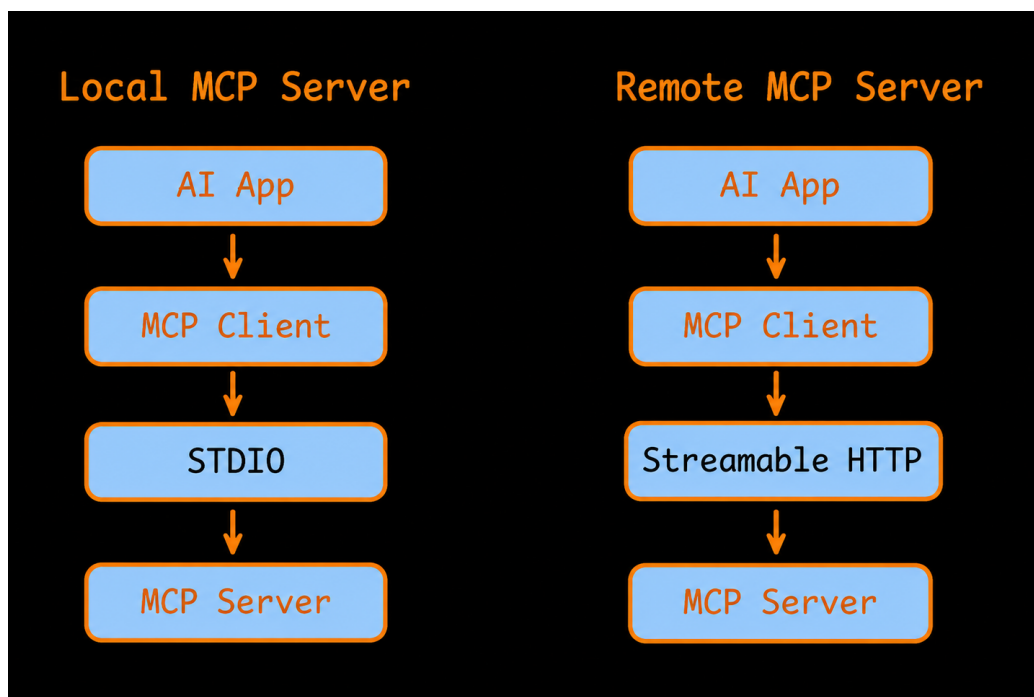
```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "temperature": "38°C",
    "condition": "Sunny"
  }
}
```

SECTION 7

Types of MCP Servers

MCP servers 3 tarah ke hote hain:

- **Local MCP Server** — same machine par run hota hai jaha aapki AI application chal rahi hai.
- **Remote MCP Server** — internet par kisi doosri machine (cloud) par run hota hai.
- **Custom MCP Server** — jab koi ready-made server available na ho, to aap khud apna MCP Server bana sakte hain (jaise ke video mein Weather MCP Server khud bana kar dikhaya gaya).



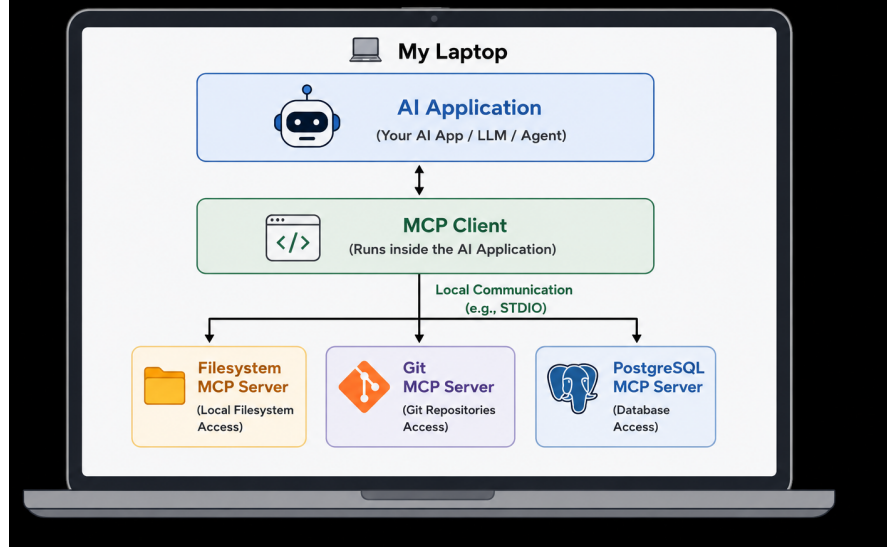
Local MCP Server communication STDIO se hoti hai, jabke Remote MCP Server communication Streamable HTTP se hoti hai.

Local MCP Server

Local MCP Server usi laptop/machine par chalta hai jaha AI Application run ho rahi hai. Filesystem, Git, aur PostgreSQL jaise servers aksar local hote hain kyunke ye direct local resources access karte hain.

Local MCP Server

A Local MCP Server runs on the same machine where your AI application is running.

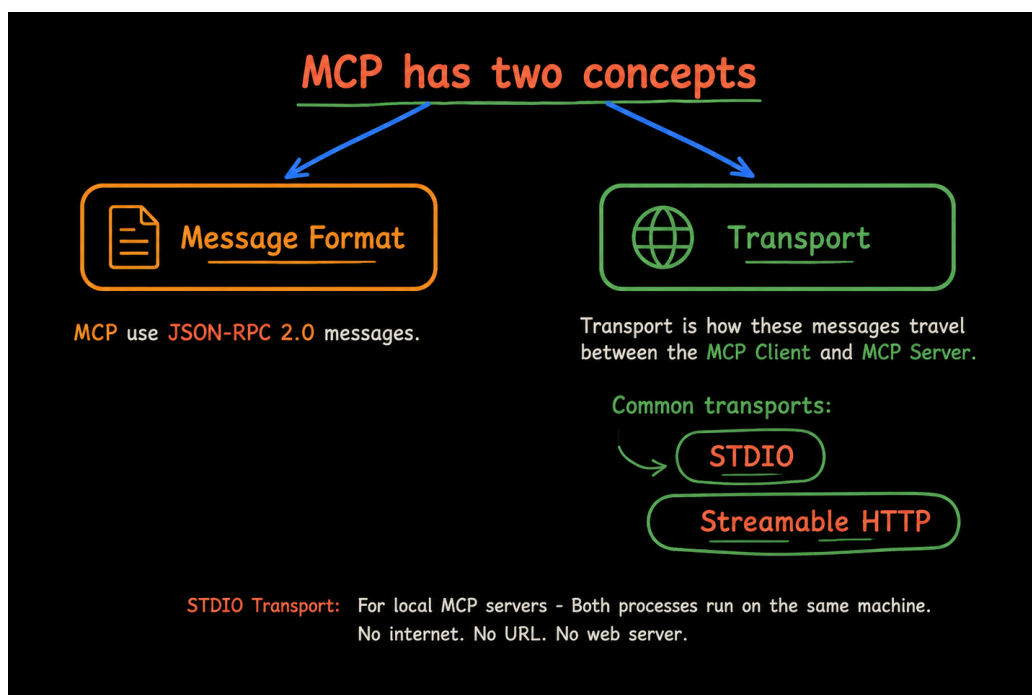


Example: My Laptop ke andar AI Application → MCP Client → Local Communication (STDIO) → Filesystem / Git / PostgreSQL MCP Servers.

SECTION 8

Transport Types — STDIO vs Streamable HTTP

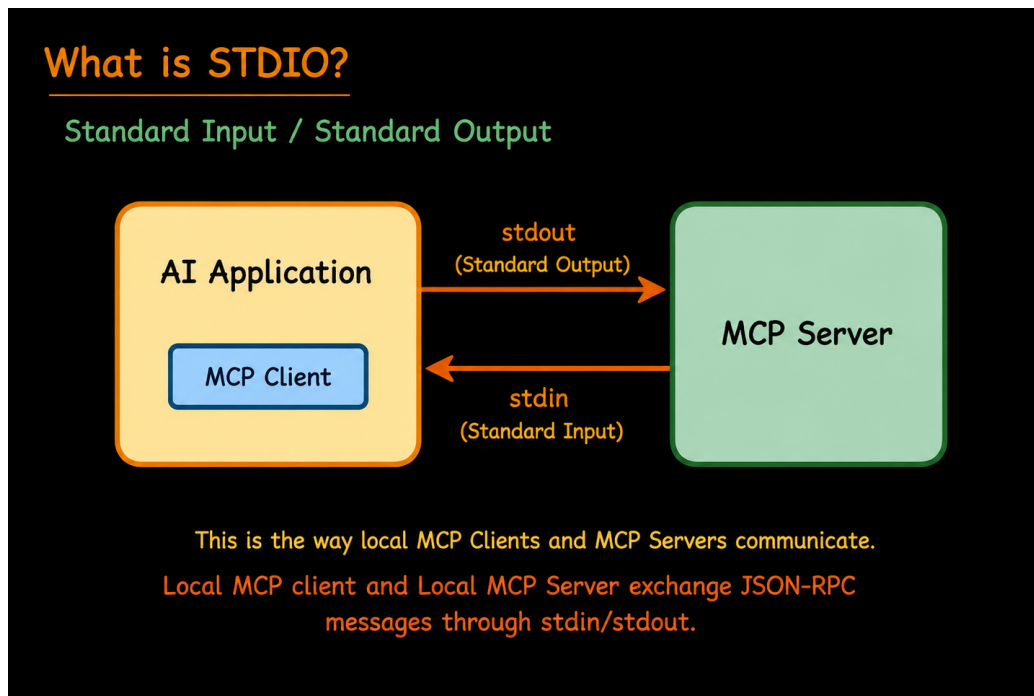
MCP ke 2 concepts hote hain: **Message Format** (JSON-RPC 2.0) aur **Transport** (messages kaise travel karte hain Client aur Server ke darmiyan).



MCP has two concepts: Message Format (JSON-RPC 2.0) aur Transport (STDIO ya Streamable HTTP).

STDIO (Standard Input / Standard Output)

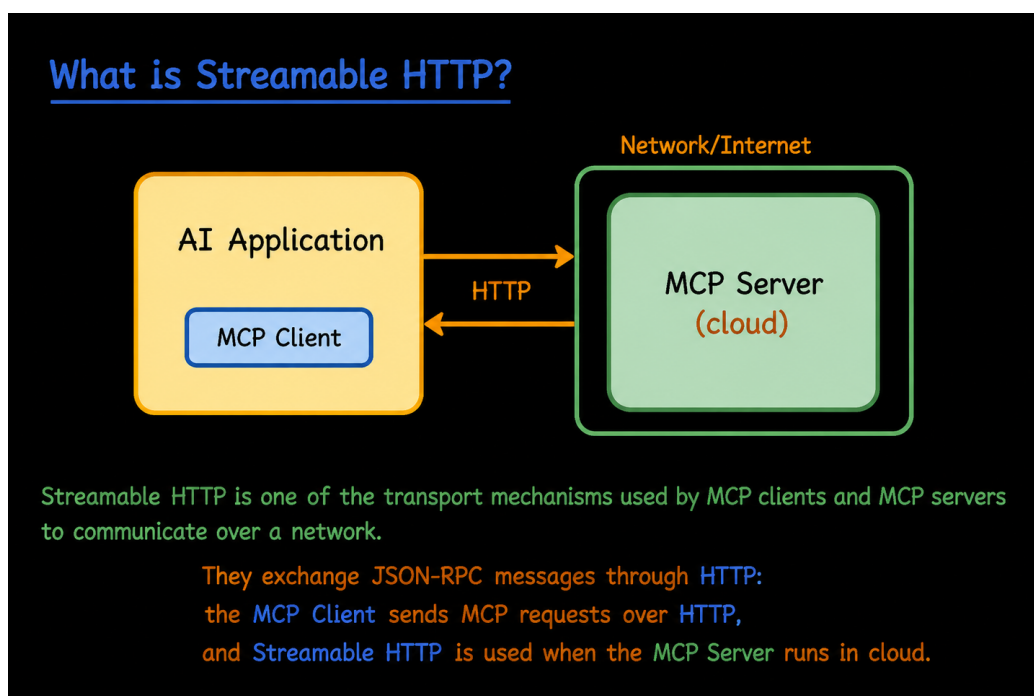
Ye **local MCP servers** ke liye use hota hai. AI Application aur MCP Server, **stdin/stdout** ke zariye JSON-RPC messages exchange karte hain. Koi internet, koi URL, koi web server ki zaroorat nahi — dono processes same machine par chal rahe hote hain.



AI Application (MCP Client) stdout se MCP Server ko request bhejta hai, MCP Server stdin se response wapas bhejta hai.

Streamable HTTP

Ye **remote MCP servers** (cloud par chalne wale) ke liye use hota hai. MCP Client, HTTP requests ke zariye Network/Internet par MCP Server se baat karta hai.



AI Application (MCP Client) HTTP ke zariye Network/Internet par cloud mein chal rahe MCP Server se JSON-RPC messages exchange karta hai.

SECTION 9

Practical Implementation — TripMate AI Project

Video mein practically dikhaya gaya ke TripMate AI (multi-agent travel planner, LangGraph + LangSmith + PostgreSQL ke sath) mein teeno types ke MCP servers integrate kiye gaye: **Remote (Tavly)**, **Local (Aviation Stack)**, aur **Custom (Weather)**.

Step 1 — Required Libraries

- **langchain-mcp-adapters** — MCP Client banane ke liye (LangChain ka pre-built MCP client wrapper).
- **nest-asyncio** — nested event loops handle karne ke liye (kyunke FastAPI backend aur MCP dono asynchronous programming use karte hain, is se "asyncio.run cannot be called from a running event loop" wala error aata hai jo nest_asyncio.apply() se fix hota hai).
- **mcp** — custom MCP Server banane ke liye Python library.

```
pip install langchain-mcp-adapters nest-asyncio mcp
```

Step 2 — Remote MCP Server (Tavly Search)

Tavly ek remote MCP server provide karta hai (internet search ke liye). Isko MultiServerMCPClient mein add karte waqt transport "**streamable_http**" rakha jata hai.

```
from langchain_mcp_adapters.client import MultiServerMCPClient
import nest_asyncio, asyncio

client = MultiServerMCPClient({
    "tavily": {
        "transport": "streamable_http",
        "url": "https://mcp.tavily.com/mcp/?tavilyApiKey=<TAVILY_API_KEY>"
    }
})

async def get_all_tools():
    tools = await client.get_tools()
    for tool in tools:
        print(tool.name)

asyncio.run(get_all_tools())
```

Step 3 — Local MCP Server (Aviation Stack)

Aviation Stack ka koi remote MCP server available nahi tha, is liye GitHub se ek open-source local MCP server use kiya gaya. Isme transport "**stdio**" hota hai, aur command uvx (ya python) ke zariye server ko run kiya jata hai.

```
client = MultiServerMCPClient({
    "tavily": { ... },
    "aviationstack": {
        "transport": "stdio",
        "command": "uvx",
        "args": ["aviation-stack-mcp"],
        "env": {"AVIATIONSTACK_API_KEY": "<AVIATIONSTACK_API_KEY>"}
    }
})

# Available tools: flights_by_airline, flight_arrival_departure_schedule,
# list_airports, list_airlines, list_routes, etc.
```

Step 4 — Custom MCP Server (Weather)

Jab koi ready-made server available na ho, khud apna MCP server banaya ja sakta hai. FastMCP class se server define karke, kisi bhi Python function par `@mcp.tool()` decorator laga kar use MCP tool bana diya jata hai.

```
from mcp.server.fastmcp import FastMCP
import requests, os

mcp = FastMCP("Weather MCP Server")
OPENWEATHER_API_KEY = os.getenv("OPENWEATHER_API_KEY")

@mcp.tool()
def get_current_weather(city: str):
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {"q": city, "appid": OPENWEATHER_API_KEY, "units": "metric"}
    response = requests.get(url, params=params)
    if response.status_code != 200:
        return response.json()
    data = response.json()
    return {
        "city": city,
        "temperature": data["main"]["temp"],
        "feels_like": data["main"]["feels_like"],
        "humidity": data["main"]["humidity"],
        "condition": data["weather"][0]["description"],
        "wind_speed": data["wind"]["speed"],
    }

@mcp.tool()
def get_forecast(city: str):
    # similar API call to forecast endpoint
    ...

if __name__ == "__main__":
    mcp.run() # runs locally, transport = stdio
```

Is custom server ko client mein add karte waqt transport **"stdio"** hota hai aur command mein Python interpreter ka path + server file ka path diya jata hai.

```
client = MultiServerMCPClient({
    "tavily": { ... },
    "aviationstack": { ... },
    "weather": {
        "transport": "stdio",
        "command": "C:/.../envs/travel/python.exe",
        "args": ["C:/.../custom_weather_mcp_server.py"],
        "env": {"OPENWEATHER_API_KEY": "<OPENWEATHER_API_KEY>"}
    }
})
```

Step 5 — Agent Ke Andar Tool Use Karna

Har agent (Hotel Agent, Flight Agent, Weather Agent) apni zaroorat ke mutabiq specific MCP tool nikaal kar (get_tools() se filter kar ke) usko invoke karta hai, aur result ko LLM ke sath combine kar ke final response banata hai.

```
async def weather_mcp_search(city: str):
    tools = await client.get_tools()
    weather_tool = next(t for t in tools if t.name == "get_current_weather")
    forecast_tool = next(t for t in tools if t.name == "get_forecast")
    weather_data = await weather_tool.ainvoke({"city": city})
    forecast_data = await forecast_tool.ainvoke({"city": city})
    return weather_data, forecast_data

# Weather Agent node (LangGraph)
def weather_agent(state):
    city = extract_destination(state["user_query"])
    weather, forecast = asyncio.run(weather_mcp_search(city))
    state["weather_result"] = f"{weather}\n{forecast}"
    return state
```

Result: TripMate AI ab flight info, hotel suggestions, weather info, day-by-day itinerary, estimated budget aur final recommendation — sab kuch MCP servers ke zariye generate karta hai, bina kisi hardcoded custom API function ke.

SECTION 10

Benefits Summary — Without MCP vs With MCP

Pehlu (Aspect)	Without MCP	With MCP
API version change	Manually code update karna parta hai	MCP Server khud upgrade handle karta hai
Framework switch (LangGraph → CrewAI)	Har tool dobara likhna parta hai	MCP connection wahi rehta hai, kuch change nahi
Maintenance	Har API documentation manually check karni parti hai	Third-party MCP servers khud maintain hote hain
Scalability	Hazon tools manage karna mushkil	Reusable servers — ek dafa banao, har jaga use karo
Downtime risk	API change hone par app break ho sakti hai	Self-healing — koi downtime nahi

MCP Kab Use Karein?

- Jab AI agent ko kisi **external tool/API/database** se baat karni ho.
- Jab aap **multiple frameworks** (LangGraph, CrewAI, OpenAI SDK) ke beech switch karne ka socho.
- Jab tool/API **frequently update** hoti rehti ho.
- Jab aap chahte ho ke aapki application **production-grade aur maintainable** ho.

Agar aap tools use hi nahi kar rahe (sirf plain LLM chat), to us waqt MCP ki zaroorat nahi hoti.

SECTION 11

Extra Additions — Agentic AI Engineer Ke Liye

Neeche wo points hain jo original video mein directly nahi thay lekin ek AI/Agentic AI Engineer ke perspective se important hain — practical use mein kaam aane wali baatein.

1. Popular Ready-Made MCP Servers (Community)

- **Filesystem MCP Server** — local files read/write.
- **GitHub MCP Server** — repos, issues, PRs manage karna.
- **Slack MCP Server** — messages bhejna/parhna.
- **Postgres / MySQL MCP Server** — database query karna.
- **Puppeteer / Browser MCP Server** — web automation / scraping.
- **Google Maps MCP Server** — location aur directions.
- In sab ki list **MCP Market** jaisi websites par mil jati hai — koi bhi naya server banane se pehle wahan check kar lena chahiye taake dobara kaam na karna paray.

2. MCP Security Best Practices

- API keys ko kabhi bhi code mein hardcode na karein — **environment variables (.env)** use karein.
- Remote MCP server use karte waqt hamesha **HTTPS** aur authentication (API key / OAuth) confirm karein.
- Agar custom MCP server bana rahe hain to **input validation** zaroor lagayein (tool arguments par).
- Production mein MCP server ko **rate limiting** aur **logging** ke sath deploy karein.

3. Debugging Tips

- Agar "asyncio.run() cannot be called from a running event loop" error aaye — `nest_asyncio.apply()` call karein app start hone se pehle.
- Agar tool call fail ho raha ho — pehle `client.get_tools()` print kar ke check karein ke tool ka naam sahi match ho raha hai ya nahi.
- Local (stdio) MCP server test karne ke liye pehle terminal mein standalone chala kar dekh lein ke server properly start ho raha hai.

4. MCP vs Traditional Function-Calling — Quick Comparison

Traditional Tool/Function Calling	MCP (Model Context Protocol)
Har framework ke liye tool alag se likhna parta hai	Ek MCP server, kisi bhi framework ke sath compatible
Tool code aapki app ke andar hi rehta hai	Tool logic separate, reusable server mein rehta hai
API update par manual fix	Third-party server khud update ho jata hai

Sharing mushkil (copy-paste karna parta hai)

Sharing asaan — server URL ya package share karo

5. One-Line Summary (Interview Ready)

"MCP ek open standard protocol hai jo AI applications (LLMs/Agents) ko external tools, APIs, aur data sources ke sath standardized tareeqe se connect karta hai — using MCP Client, MCP Server, aur JSON-RPC 2.0 messages, jo STDIO (local) ya Streamable HTTP (remote) transport ke zariye communicate karte hain."

Notes Roman Urdu mein original video transcript aur uploaded Excalidraw diagram file se compile ki gayi hain — AI Engineer / Agentic AI Engineer ke revision aur reference ke liye.